

Documentation technique — Vite & Gourmand

Auteur : Dylan Quinzin

Formation : Titre Professionnel Développeur Web et Web Mobile (RNCP 37674) — Studi

Cadre : Évaluation en Cours de Formation (ECF)

Dépôt GitHub : <https://github.com/dquinzin1-jpg/vite-et-gourmand>

Application déployée : <https://dp-vite-et-gourmand.alwaysdata.net>

1. Réflexions techniques initiales

Le sujet impose une seule contrainte technologique : utiliser à la fois une base de données **relationnelle** et une base de données **non relationnelle**. Le reste de la stack était libre. J'ai donc retenu une stack serveur classique et bien documentée, adaptée au référentiel TP DWWM et à un projet de cette taille.

Choix technologiques et justifications

- **PHP 8 (avec PDO)** : langage côté serveur le plus répandu pour les sites dynamiques, gratuit et au cœur du référentiel. J'utilise l'extension **PDO** plutôt que **mysqli**, car elle est plus moderne, portable entre SGBD, et surtout sécurisée grâce aux requêtes préparées qui protègent des injections SQL.
- **MariaDB** (base relationnelle) : fork open source de MySQL, c'est la **source de vérité transactionnelle** du projet (comptes, menus, commandes). MySQL en local (XAMPP), MariaDB 11.4 en production (AlwaysData), le SQL étant compatible entre les deux.
- **MongoDB Atlas** (base non relationnelle) : utilisée comme **base analytique** pour alimenter le graphique de statistiques de l'espace administrateur. Ce choix est détaillé dans la section sur l'architecture hybride.
- **Bootstrap 5** : framework CSS pour un design responsive professionnel rapidement, sans réinventer la grille et les composants.
- **JavaScript (vanilla)** : pour les interactions côté client sans rechargement (filtres dynamiques de la page menus, calcul du prix en temps réel sur la page commande).
- **Chart.js** : bibliothèque légère pour afficher les statistiques de vente sous forme de graphique en barres.
- **Git / GitHub** pour le versionnage, **VS Code** comme éditeur, **AlwaysData** pour l'hébergement, **FileZilla** (SFTP) pour le déploiement.

2. Configuration de l'environnement de travail

Le projet repose sur deux environnements : un environnement **local** de développement et un environnement **de production** en ligne.

Environnement local (Windows 11)

- **XAMPP** fournit Apache, MySQL/MariaDB et PHP en une seule installation. Le code est placé dans `C:\xampp\htdocs\vite-et-gourmand`, accessible via `http://localhost/vite-et-gourmand`.
- **VS Code** comme éditeur (extensions PHP, terminal intégré pour Git).
- **Opera** comme navigateur de test, avec les outils de développement (F12) pour déboguer.

Environnement de production (AlwaysData)

- Serveur **PHP 8.4** + base **MariaDB 11.4**.
- Déploiement des fichiers par **SFTP** (FileZilla), import de la base via **phpMyAdmin**.

Détection automatique de l'environnement

Un point central du projet est le fichier `config/db.php`, qui **détecte automatiquement l'environnement** et adapte la connexion sans modification manuelle du code. Il lit les identifiants depuis un fichier `.env` (jamais versionné) et bascule sur la bonne configuration selon le nom d'hôte :

```
if (isset($_SERVER['HTTP_HOST'])
    && strpos($_SERVER['HTTP_HOST'], 'alwaysdata.net') !== false) {
    // Configuration EN LIGNE (AlwaysData)
} else {
    // Configuration LOCALE (XAMPP)
}

$pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $username, $password);
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
```

La connexion PDO est configurée en `utf8mb4` (support complet des accents et emojis), en mode **exception** (les erreurs SQL lèvent une exception au lieu d'échouer silencieusement) et en mode de récupération **associatif** par défaut. De la même façon, une variable `BASE_URL` définie dans l'en-tête s'adapte au sous-dossier en local et à la racine en production, ce qui garantit des liens corrects dans les deux environnements.

3. Architecture hybride : relationnel + non relationnel

L'application combine deux bases de données aux rôles complémentaires :

- **MariaDB** est la **source de vérité transactionnelle** : tout ce qui doit être fiable et cohérent (comptes, menus, commandes, suivi) y est stocké, avec des transactions SQL.
- **MongoDB Atlas** est une **base analytique** : à chaque commande validée, un document résumé y est inséré. L'espace administrateur interroge ensuite MongoDB via un *pipeline* d'agrégation (`$match` / `$group` / `$sort`) pour produire les statistiques de vente affichées avec Chart.js.

L'insertion dans MongoDB se fait **après** la validation (commit) de la transaction MariaDB, dans un bloc `try/catch` non bloquant : si MongoDB échoue, la commande reste valide. Cela évite qu'une indisponibilité de la base analytique n'empêche un client de commander.

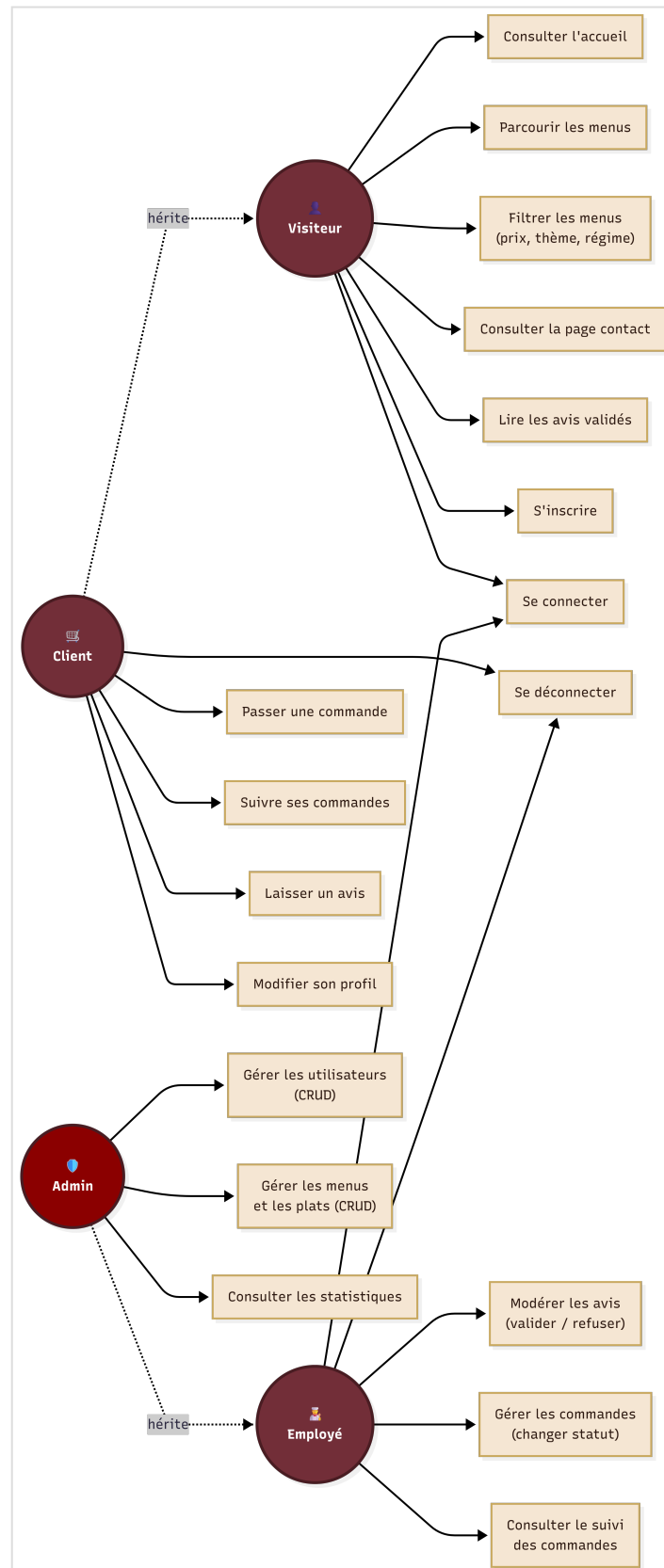
Limite identifiée (et axe d'amélioration) : l'extension PHP MongoDB fonctionne en local mais, sur le serveur AlwaysData, elle se charge en ligne de commande (CLI) mais **pas en mode Web** (configuration Apache cgi-fcgi). Après diagnostic, j'ai fait le choix pragmatique de conserver le fonctionnement MongoDB en local et de documenter cette limite. Une amélioration future consisterait à interroger MongoDB Atlas via son API HTTPS (sans dépendre de l'extension PHP) ou à choisir un hébergement la supportant en mode Web.

4. Modèle de données

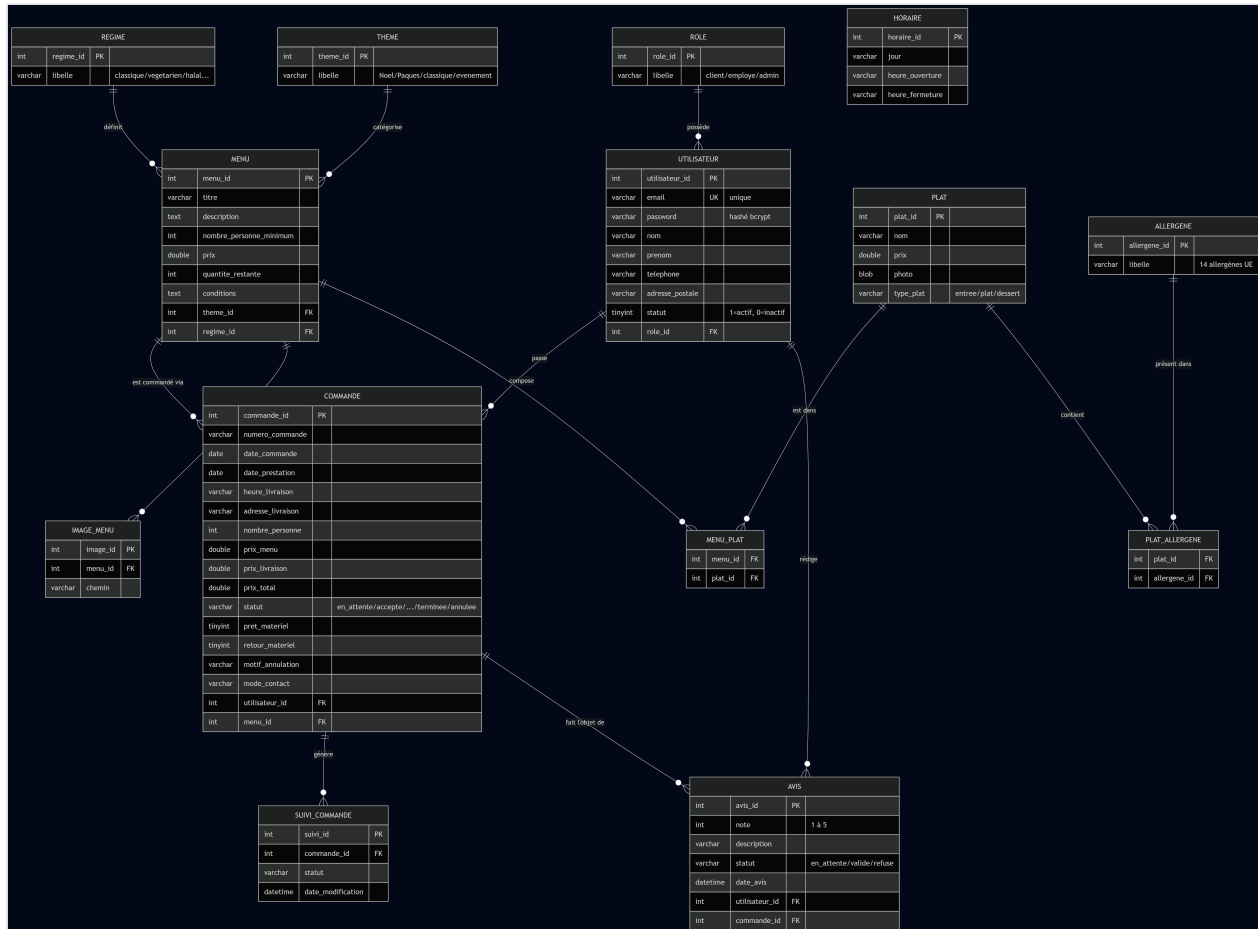
La base relationnelle compte **14 tables**, avec une convention de nommage homogène (clé primaire `xxx_id`). Les relations « plusieurs-à-plusieurs » sont gérées par des **tables d'association** : `menu_plat` (un plat peut figurer dans plusieurs menus et inversement) et `plat_allergene` (un plat peut contenir plusieurs allergènes). La gestion des rôles repose sur la table `role` (administrateur, employé, utilisateur), et l'historique des statuts d'une commande est conservé dans `suivi_commande`.

Liste des tables : `role`, `utilisateur`, `regime`, `theme`, `menu`, `allergene`, `plat`, `plat_allergene`, `menu_plat`, `image_menu`, `commande`, `suivi_commande`, `avis`, `horaire`.

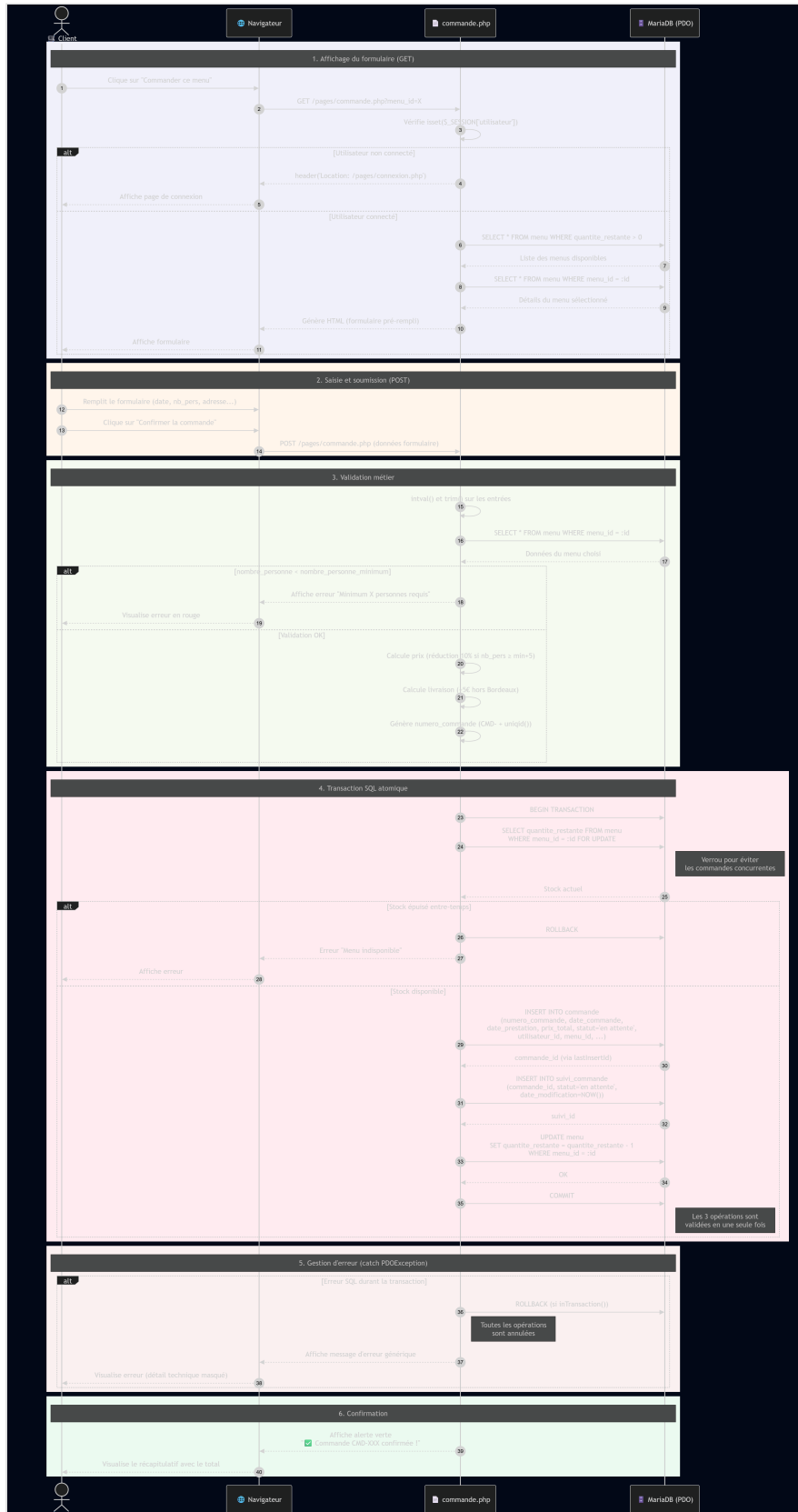
4.1 Diagramme de cas d'utilisation



4.2 Diagramme de classes



4.3 Diagramme de séquence



5. Logique métier : le parcours de commande

La création d'une commande est l'opération la plus sensible du site : elle touche plusieurs tables et ne doit jamais laisser la base dans un état incohérent. Elle est donc encadrée par une **transaction SQL atomique** :

1. Ouverture de la transaction (`beginTransaction`).
2. Re-vérification du stock avec un **verrou** (`SELECT ... FOR UPDATE`) pour empêcher la survente en cas de commandes simultanées (*race condition*).
3. Insertion de la commande (`commande`), puis du premier état dans `suivi_commande`, puis décrémentation du stock du menu.
4. Validation de l'ensemble (`commit`) — ou annulation complète (`rollback`) si la moindre étape échoue.

Le calcul du prix applique les règles métier du sujet : prix au prorata du nombre de personnes, **réduction de 10 %** à partir de « minimum + 5 » personnes, et **frais de livraison** hors Bordeaux (5 € + 0,59 €/km). Une fois la commande validée, le document analytique est inséré dans MongoDB, puis un e-mail de confirmation est généré.

6. Sécurité

Plusieurs mécanismes protègent l'application, répartis entre le front-end, le back-end et la base de données.

- **Mots de passe** : hachés avec `password_hash()` et l'algorithme **bcrypt** ; jamais stockés en clair. Vérification à la connexion avec `password_verify()`.
- **Injections SQL** : toutes les requêtes utilisent des **requêtes préparées PDO** (`prepare()` / `execute()`), qui séparent les données des commandes SQL.
- **Sessions** : `session_regenerate_id(true)` à chaque connexion réussie pour prévenir la fixation de session ; vérification systématique du rôle sur chaque page protégée.
- **XSS** : toutes les données affichées passent par `htmlspecialchars()`.
- **Validation côté serveur** : toutes les entrées des formulaires sont validées côté serveur (type, longueur, format via `filter_var()`), en plus de la validation HTML5 côté client.
- **Gestion des secrets** : les identifiants de connexion sont stockés dans un fichier `.env` **exclu de Git** via `.gitignore`. Au cours du projet, j'ai détecté que le `.env` avait été versionné par erreur : je l'ai retiré du suivi (`git rm --cached`) et **régénéré tous les mots de passe** concernés.
- **Création de comptes** : un visiteur ne peut s'inscrire que comme client ; les comptes employés sont créés uniquement par un administrateur, et aucun compte administrateur ne peut être créé depuis l'application.

7. Documentation du déploiement

Le déploiement sur AlwaysData a suivi les étapes ci-dessous :

1. **Création du compte et des services** sur AlwaysData : un site PHP 8.4 et une base de données MariaDB 11.4.
2. **Transfert des fichiers** du projet via **FileZilla** en **SFTP** (hôte `ssh-dp-vite-et-gourmand.alwaysdata.net` , port 22) vers le dossier `/home/dp-vite-et-gourmand/www/` .
3. **Import de la base** : exécution du script `database/database.sql` (structure) puis de `database/seed-data.sql` (données de démonstration) via phpMyAdmin côté AlwaysData.
4. **Configuration de l'environnement** : renseignement du fichier `.env` côté serveur avec les identifiants de la base distante. La détection automatique de `config/db.php` bascule alors seule sur la configuration de production.
5. **Extension MongoDB** : compilation via PECL sur le serveur. Elle fonctionne en CLI mais pas en mode Web (voir la limite documentée en section 3) ; les statistiques MongoDB sont donc opérationnelles en environnement local.
6. **Mises à jour ultérieures** : à chaque évolution testée en local, les fichiers concernés sont renvoyés par SFTP, et le code est versionné sur GitHub (branches `feature/*` → `dev` → `main`).

L'application est accessible publiquement à l'adresse <https://dp-vite-et-gourmand.alwaysdata.net>.